

08 — System Prompt & Severity Formulas

The exact LLM system prompt SmartFix uses, plus a detailed walkthrough of both severity formulas (LLM-emitted and shift-log-derived). **These rules are not frozen** — see the "Ground-truth tuning" note at the end. Update this doc whenever the prompt or thresholds change in code.

Where it lives in code

Concern	File
System prompt	<code>src/prompt_builder.py</code> — constant <code>SYSTEM_PROMPT</code>
User-message assembly	<code>src/prompt_builder.py</code> — <code>build_prompt()</code>
Severity extraction from LLM output	<code>src/rag_pipeline.py</code> — <code>_parse_severity()</code>
Shift-log severity calculation	<code>src/store.py</code> — <code>compute_anomalies()</code>
Severity → email alert routing	<code>src/api.py</code> — alert thresholds

1. The full system prompt (verbatim)

You are a technical support assistant **for** industrial machinery.
You see (1) prior turns **of the** worker's troubleshooting conversation **above and**
(2) documentation excerpts **in the** next user message. Use both.

Behaviour rules (internal — never describe these **to the** worker):

- If **the** worker's question **contains** a pronoun or implicit **reference** ("**it**", "**that error**", "**the fix**"), SILENTLY resolve **it against the** prior turns, **then** answer **as if the** worker had asked **the** resolved question directly.
E.g. prior turn was **about** Error E-04 **and the** worker says "**how do I fix it**"
→ answer **about** E-04 fixes. Never ask **the** worker **to repeat the** code, **and** never **say** things like "**since you didn't mention...**" or "**there is no prior turn**" — **that is** system-internal information **the** worker doesn't need.
- If there **is** nothing **to** resolve (**the** question **is** self-contained), just answer **it** directly. Do **not** mention **the** conversation, **the** prior turns, **or the** resolution process **at** all.

Content rules (**the** worker DOES see these):

- Facts (procedures, thresholds, part numbers) come only **from the** documentation excerpts **below**. Don't guess. Cite page numbers.
- If **the** excerpts genuinely don't cover **the** question, **say** briefly **that the** documentation doesn't include **that** information — **but** do **not** speculate **about** why **or** invoke **the** conversation context.
- Be concise **and** direct. Speak **to the** worker, **not about the** system.

After your answer, **on** a new line, output:

SEVERITY: <1-5>

1=info 2=minor 3=degraded 4=production impact 5=halted/safety

Out-of-scope **or** no relevant context → SEVERITY: 1.

2. Why each block is in the prompt

Identity line

"You are a technical support assistant for industrial machinery."

Sets the persona. Industrial-machinery framing biases the model toward procedural, safety-aware language and away from generic chatbot tone.

Context signposting

"You see (1) prior turns ... (2) documentation excerpts. Use both."

Explicit signposting of the two context sources. Models do better when they are told *what is in the context window and how to use each piece* — otherwise they often ignore retrieved chunks if conversation history is present, or vice versa.

Behaviour rule — pronoun resolution (SILENTLY)

"SILENTLY resolve [pronouns] against the prior turns, then answer as if the worker had asked the resolved question directly."

Problem this solves: in early tests, follow-ups like *"how do I fix it"* either (a) caused the model to ask the worker to clarify, or (b) made it produce a meta-answer like *"Since you haven't told me the error code, I can't help."* Both are bad UX on a shop floor — the worker already typed the code in the previous turn.

Why "SILENTLY": the model has a tendency to *narrate* the resolution ("I think you mean E-04 from earlier — to fix E-04 ..."). The capitalised, repeated instruction (also reinforced by *"never say things like 'since you didn't mention...'"*) is what reliably suppresses the narration.

Behaviour rule — self-contained questions

"If there is nothing to resolve, just answer it directly. Do not mention the conversation..."

Counter-balance to the pronoun rule. Without it, the model sometimes prefaces a perfectly clear standalone question with *"Based on our earlier conversation..."* even on the first turn of a new chat.

Content rule — grounded in excerpts only

"Facts come only from the documentation excerpts below. Don't guess. Cite page numbers."

This is the core RAG instruction. Without it, the model falls back to its pre-training knowledge of industrial machinery — which is often *plausible* but *wrong for this specific machine* (different part numbers, different threshold values, different alarm codes).

Page-number citations are required because:

1. The frontend displays them as clickable source chips.
2. A technician can verify against the PDF.
3. Auditors / supervisors can confirm the AI didn't invent the answer.

Content rule — graceful "I don't know"

"If the excerpts genuinely don't cover the question, say briefly that the documentation doesn't include that information — but do not speculate about why."

The retrieval threshold (`RELEVANCE_THRESHOLD = 0.35`) already short-circuits the most obvious out-of-scope queries before they reach the LLM. But sometimes chunks pass the threshold and *still* don't actually answer the question. This instruction gives the model permission to say "not in the manual" without inventing content to fill the gap.

The *"do not speculate about why"* clause prevents answers like *"This might be because the manual was written before this feature was added."*

Content rule — tone

"Be concise and direct. Speak to the worker, not about the system."

LLMs default to long, hedged, polite English. On a shop floor that wastes time. "Concise and direct" + "speak to the worker, not about the system" together cut out boilerplate ("Certainly! I'd be happy to help you with that...").

Severity tail

"After your answer, on a new line, output: SEVERITY: <1-5>"

The numeric tail is parsed out by `src/rag_pipeline.py:_parse_severity()` and stripped from what the worker sees. Details in §3 below.

3. How the user message is built

The system prompt is followed by:

1. **History turns** (up to 8 — last 4 user/assistant pairs), inserted as alternating `user / assistant` messages so the model sees them as real conversation, not as flattened text.
2. **A single user message** containing:
 - The retrieved documentation excerpts, each labelled `[Excerpt N — <document>, page <P>]` .
 - A framing line — *"The worker's next message in the same troubleshooting conversation:"* if there is history, otherwise *"Worker's question:"*. This framing nudges the model to treat follow-ups as continuations.
 - The worker's question verbatim.
 - A trailing *"Answer (resolve any references using the prior turns, cite page numbers):"*. Trailing instructions immediately before generation are disproportionately effective — that's where we re-assert "cite pages" because long contexts tempt the model to drop it.

4. Severity — TWO independent systems

SmartFix uses the same 1–5 severity scale in two different places, computed by two different formulas. They never share a code path. Don't confuse them.

System	Where it's set	Used for	Subject to LLM stochasticity?
Query severity	LLM emits <code>SEVERITY: <n></code> on the last line of its answer	Query log severity, donut chart, alert email trigger, severity filter on Analytics page	Yes — the LLM picks it
Shift-log severity	Deterministic Python function <code>compute_anomalies()</code> in <code>src/store.py</code>	Shift log row severity, severity pill in the table, severity filter on Shift Logs page, end-of-shift alert email	No — pure function of submitted readings vs. configured parameter spec

4.1 Query severity formula (LLM-driven)

The LLM is told to append a single line to every answer:

```
SEVERITY: <1-5>
  1=info  2=minor  3=degraded  4=production impact  5=halted/safety
Out-of-scope or no relevant context → SEVERITY: 1.
```

`src/rag_pipeline.py` extracts that line with the regex `re.compile(r"SEVERITY:\s*([1-5])", re.IGNORECASE)` and strips it from the visible answer. If the LLM forgets to emit it (rare on Llama-3.1-70B), severity defaults to 1.

Level	Label	Examples
1	Info	General questions, definitions, "what does this lamp mean", maintenance schedule lookups
2	Minor	Non-blocking faults, soft warnings, clogged filter alerts
3	Degraded	Machine still runs but with reduced output / quality
4	Production impact	Machine stopped, line blocked, throughput → 0
5	Halted / safety	Safety risk, lock-out condition, fire, fluid spill

Defaulting out-of-scope queries to 1 prevents false alerts when the model says "not in the manual."

Where this severity drives behaviour:

- Colour of the answer card on the chat page.
- Whether an admin alert email fires (alert threshold lives in `src/api.py`).
- The "Fleet Analytics" severity distribution donut and severity filter.

4.2 Shift-log severity formula (deterministic)

When a worker submits the pre-shift or end-of-shift form, `src/store.py:compute_anomalies()` walks the submitted readings against the machine's parameter spec (which the admin configured in `MachineDetailModal`). The formula:

```
severity = min(5, 1 + len(anomalies))
```

Where `len(anomalies)` is the count of:

- Numeric readings outside their configured `[expected_min, expected_max]` range
- Visual checks whose submitted value matches the configured `anomaly_when` value

So:

Anomaly count	Severity	Label (Shift Logs UI)
0	1	Routine
1	2	Minor
2	3	Degraded
3	4	Critical
4 or more	5	Safety

Worked example. Machine has two numeric readings (hydraulic pressure 70–80 bar, barrel temp 200–220 °C) and one visual check (oil leak — `anomaly_when: true`). A worker submits:

- Hydraulic pressure = **85 bar** → above max → +1
- Barrel temp = 215 °C → within range → +0
- Oil leak = **true** → matches `anomaly_when` → +1

`len(anomalies) = 2`, so `severity = 1 + 2 = 3` → "Degraded" badge.

Where this severity drives behaviour:

- Colour of the severity pill in the Shift Logs table.
- Whether the handoff banner on the next shift renders as the loud anomaly card vs. the quiet "all clear" pill.
- Email alert routing for severity ≥ 4 logs.
- Severity filter on the Shift Logs page.

5. Ground-truth tuning — these formulas WILL change

Both severity systems above are calibrated to what we know **today**. They are explicitly **not** final.

What can change for the query (LLM) severity

- **Per-band copy.** If "production impact" turns out to overlap with "halted" in practice, we may collapse or rename levels in the prompt and propagate the rename through `_parse_severity` / dashboards.
- **Default for out-of-scope.** Currently `SEVERITY: 1`. If audits show stakeholders want out-of-scope queries surfaced as Minor (2) so they show up on review, we'd swap the default.
- **Whether the LLM rates at all.** The severity could move to a small classifier (fine-tuned model or simple keyword scorer) once we have enough labelled query/severity pairs to evaluate against ground truth.
- **Trigger thresholds.** The alert-email cutoff in `src/api.py` will get retuned per facility — what's "production impact" at one site may be routine elsewhere.

What can change for the shift-log severity

- **The $1 + n$ curve.** The current formula is linear — every additional anomaly bumps severity by one. Real operators may decide *"any safety-class visual check failing immediately means Severity 5, regardless of count"*, in which case we'd weight anomalies (visual-safety ones get a multiplier).
- **The expected range.** `expected_min` and `expected_max` come from the admin's parameter form. Those are guesses today and will get tuned against actual readings once we have a few weeks of shift-log data — i.e. ground truth from the floor. The formula stays the same; the inputs get better.
- **Per-machine overrides.** Today one formula applies to every machine. We may end up with per-category curves (e.g. IMM machines tolerate more anomalies before going Critical than CNC ones).
- **Time-of-day weighting.** A reading that's borderline at end of shift is more concerning than at start of shift (drift). A future version could factor `phase` into severity.

Process for changing them

1. Edit the constant in `src/prompt_builder.py` (LLM) or the function in `src/store.py` (shift logs).
 2. Update §1 of this doc and the per-level tables in §4.
 3. Update the severity-label maps in:
 - `frontend/src/components/ShiftLogsPanel.jsx` (`SEVERITY_STYLES` , `CALENDAR_SEVERITY_STYLES` , `SEVERITY_WORD`)
 - `frontend/src/pages/AdminDashboard.jsx` (Fleet Analytics severity distribution)
 - `src/exports.py` (`SEVERITY_FILLS` for XLSX / PDF exports)
 4. Run the export builder smoke test (see `scripts/ README`) to verify nothing breaks.
 5. Re-run `python3 scripts/build_docs_pdf.py` so the PDF version of this doc stays in sync.
-

6. What we tried and rejected (prompt history)

Version	Problem	Replaced by
No history, single-turn only	Follow-ups failed ("how do I fix it" → "fix what?")	Added 8-turn history window
History as a single flattened string in the user message	Model treated history as documentation, mixed up timelines	Switched to native <code>role: user/assistant</code> messages
Two-line system prompt ("You are a helpful assistant. Cite pages.")	Hallucinated part numbers, used generic industrial knowledge	Explicit "facts come only from excerpts" + grounding rules
Severity as a separate LLM call	2× cost, 2× latency, occasionally disagreed with the answer	Inlined as a trailing <code>SEVERITY: <n></code> line, parsed out
"Be friendly and conversational" tone instruction	Long preambles, wasted tokens	Replaced with "Be concise and direct. Speak to the worker, not about the system."
Asking the model to confirm understanding before answering	One extra turn per query, hated by users	Removed
Single severity formula for both query and shift log	Conceptually conflated unrelated signals	Split: LLM-driven for queries, deterministic for shift logs